

Maestría en
Software

Patrones de Diseño de Software

Autor(es): Mauricio Ortiz Ochoa
mortizo@ups.edu.ec

Tema:

Unidad 02. Patrones de diseño estructurales

Tema:

Bridge

- *Bridge*
- Decorator

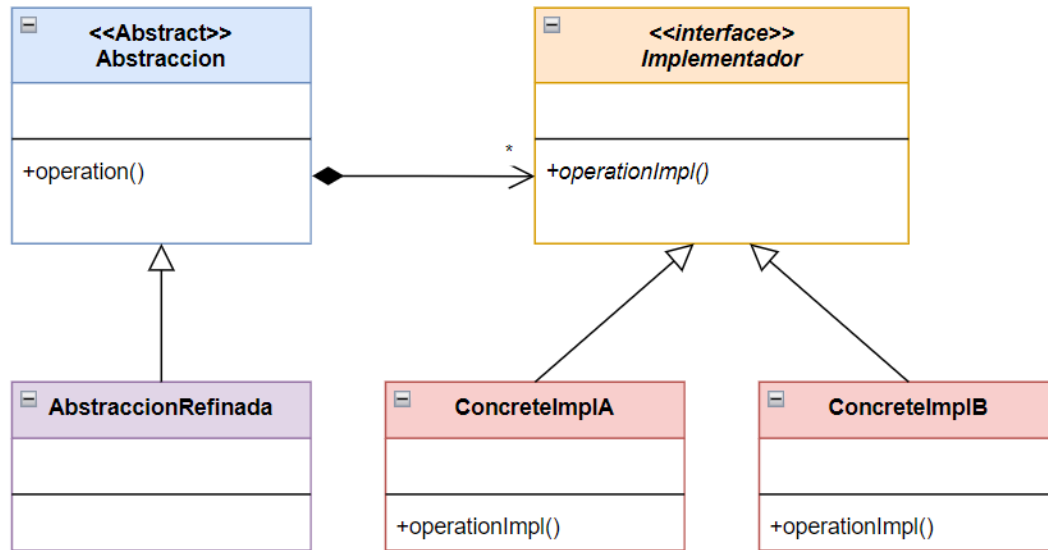
Bridge

Bridge

- Propósito
 - Desacopla una abstracción de su implementación, permitiendo que ambas puedan evolucionar de manera independiente.
 - Resulta útil cuando un diseño presenta dos dimensiones de variación que, si se representan únicamente mediante herencia, pueden producir una proliferación de subclases.
- Problema / Motivación
 - Una universidad necesita gestionar inscripciones para diferentes tipos de programas, inicialmente Grado y Posgrado.
 - Cada inscripción puede realizarse mediante distintos canales: Presencial, En línea o Móvil.
 - Si modelamos todas las combinaciones mediante herencia, aparecerían clases como:
 - inscripciongradopresencial,
 - inscripciongradoenlínea,
 - inscripciongradomóvil,
 - inscripcionposgradopresencial,
 - Inscripcionposgradoenlínea,
 - inscripcionposgradomóvil.
 - Si posteriormente incorporamos un nuevo tipo de programa, por ejemplo Tecnologías, deberíamos crear nuevamente una clase para cada canal.
 - El problema aparece porque tipo de programa y canal de inscripción son dos dimensiones que pueden cambiar independientemente.

Bridge

Estructura



- Participantes

- Abstraccion**

Define la interfaz de alto nivel que utiliza el cliente. Mantiene una referencia a un objeto de tipo Implementador y delega en él parte del trabajo.

- AbstraccionRefinada:**

Extiende o especializa la abstracción incorporando variantes de comportamiento de alto nivel.

- Implementador:**

Define el contrato para las distintas implementaciones.

Sus operaciones no tienen que coincidir exactamente con las de la Abstracción; normalmente proporcionan operaciones más específicas sobre las que la Abstracción construye su comportamiento.

- ImplementadorConcreto:**

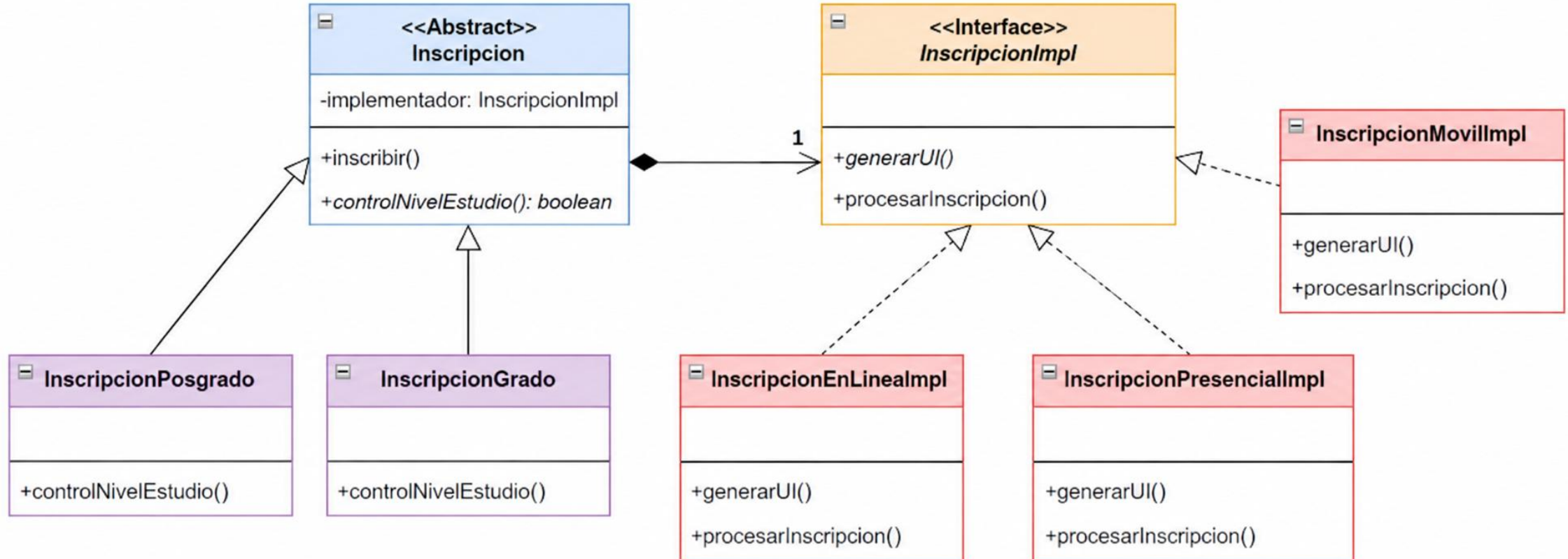
Implementa el contrato definido por Implementador y proporciona una variante concreta de implementación.

- Colaboraciones

- Cliente trabaja con la Abstracción.
 - La Abstracción mantiene una referencia al Implementador.
 - La Abstracción delega determinadas operaciones en el Implementador.
 - Las jerarquías de Abstracción e Implementador pueden evolucionar de forma independiente.

Bridge

Solución



<https://github.com/mortizo/pds-unidad02-ooms/tree/master/pds-unidad02-ooms/src/main/java/p69/bridge>

Bridge

Bridge

- Aplicabilidad
 - Cuando existen dos dimensiones de variación que deberían poder evolucionar independientemente.
 - Cuando una jerarquía empieza a crecer debido a la combinación de distintas variantes y queremos evitar una proliferación de subclases.
 - Cuando necesitamos poder cambiar o seleccionar una implementación sin modificar la abstracción que utiliza el cliente.
 - Cuando tanto la abstracción como sus implementaciones deben poder extenderse de manera independiente.
 - Cuando queremos ocultar al cliente los detalles concretos de implementación.
- Evitarlo cuando:
 - Solo existe una dimensión relevante de variación.
 - Las combinaciones son pocas, estables y no justifican introducir una segunda jerarquía.
 - La abstracción y la implementación no necesitan evolucionar independientemente.
- Bridge vs. Adapter
 - Bridge separa deliberadamente dos dimensiones de un diseño para que puedan evolucionar independientemente.
 - Adapter permite colaborar con una interfaz existente que resulta incompatible con la que necesita el cliente.

Bridge

Actividad en clase: ¿Bridge?

Caso 1 — Sistema de notificaciones

Una universidad envía distintos tipos de notificaciones: académicas, financieras y de seguridad.

Cada una puede enviarse mediante correo electrónico, SMS o notificación móvil.

¿Cómo diseñarían el sistema para incorporar nuevos tipos de notificación o nuevos canales sin crear una clase para cada combinación?

Caso 2 — Generación de reportes

Un sistema genera reportes académicos, financieros y administrativos.

Cada reporte puede producirse en PDF, Excel o HTML.

¿Cómo separarían el tipo de reporte del formato de generación para permitir que ambas dimensiones evolucionen independientemente?

Caso 3 — Controles y dispositivos

Un sistema dispone de controles básicos y avanzados que pueden manejar distintos dispositivos: televisores, proyectores y equipos de audio.

¿Cómo diseñarían el sistema para agregar nuevos controles o nuevos dispositivos sin crear una subclase por cada combinación?

Caso 4 — Visualización de gráficos

Una aplicación representa diferentes figuras, como círculos, rectángulos y líneas, utilizando distintos mecanismos de renderizado: SVG, Canvas o impresión.

¿Cómo permitirían incorporar nuevas figuras o nuevos mecanismos de renderizado de manera independiente?

Tema:

Decorator

- Bridge
- *Decorator*

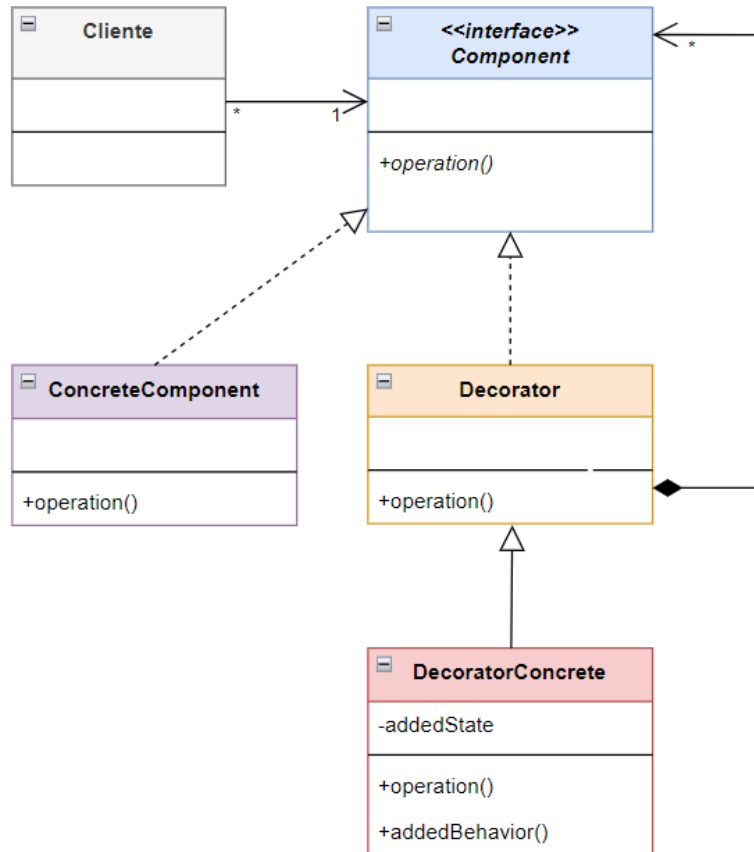
Decorator

Decorator

- Propósito
 - Asigna responsabilidades adicionales a un objeto de manera dinámica, proporcionando una alternativa flexible a la herencia para extender su funcionalidad.
 - Permite combinar responsabilidades adicionales sin crear una subclase diferente para cada combinación posible.
- Problema / Motivación
 - Un sistema de inscripciones genera una notificación básica cuando se registra una matrícula.
 - Dependiendo del contexto, la notificación puede necesitar responsabilidades adicionales, por ejemplo:
 - incorporar formato HTML;
 - agregar información para envío por correo electrónico;
 - agregar un número telefónico para SMS;
 - incorporar información de una cuenta de red social.
 - Estas funcionalidades pueden utilizarse de manera individual o combinarse según las necesidades de cada notificación.
 - Si utilizamos herencia para representar todas las combinaciones posibles, aparecerían cada vez más subclases.
 - Necesitamos poder añadir o combinar responsabilidades durante la ejecución sin modificar la clase original.

Decorator

Estructura



- Participantes

- Component**

Define la interfaz común para los objetos a los que se pueden añadir responsabilidades dinámicamente.

- ConcreteComponent**

Define el objeto base al que pueden añadirse responsabilidades adicionales.

- Decorator**

Implementa la misma interfaz que Component.

Mantiene una referencia a un objeto de tipo Component y delega en él las operaciones.

- ConcreteDecorator**

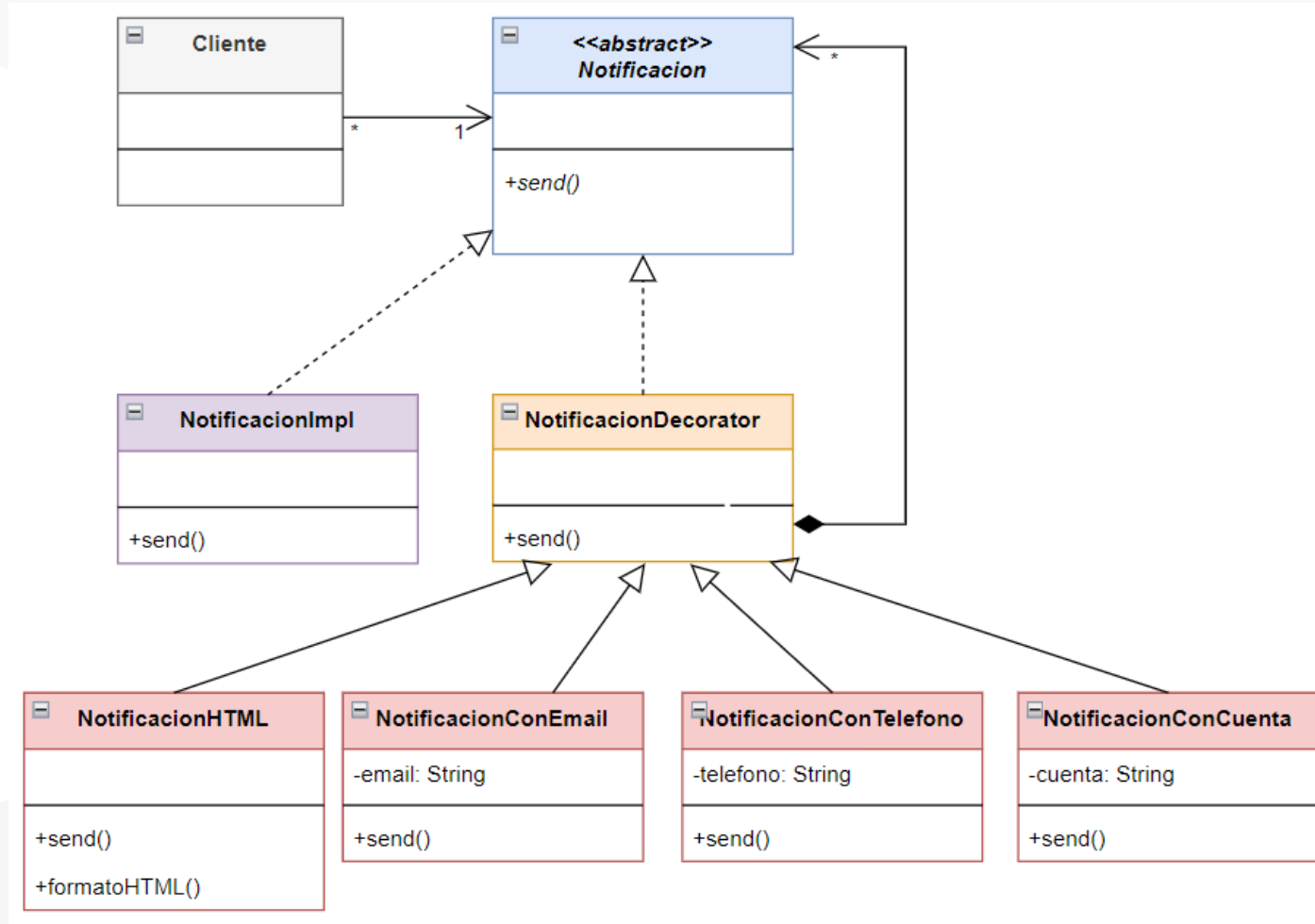
Añade una responsabilidad adicional antes o después de delegar la operación al componente envuelto.

- Colaboraciones

- El Cliente trabaja mediante la interfaz Component.
- El Decorator delega la operación al objeto Component que mantiene.
- Un ConcreteDecorator puede añadir comportamiento antes o después de la delegación.
- Como Decorator también es un Component, varios decoradores pueden encadenarse sobre el mismo objeto.

Decorator

Solución



<https://github.com/mortizo/pds-unidad02-ooms/tree/master/pds-unidad02-ooms/src/main/java/p69/decorator>

Decorator

Decorator

- Aplicabilidad
 - Cuando necesitamos añadir responsabilidades adicionales a objetos individuales de manera dinámica, sin modificar otros objetos de la misma clase.
 - Cuando las responsabilidades pueden combinarse de diferentes formas durante la ejecución.
 - Cuando algunas responsabilidades deben poder añadirse o retirarse sin modificar la clase original.
 - Cuando utilizar herencia produciría una gran cantidad de subclases para representar todas las combinaciones posibles.
 - Cuando queremos extender el comportamiento de un objeto manteniendo la misma interfaz que utiliza el cliente.
- Evitarlo cuando
 - La funcionalidad adicional es fija y no necesita combinarse dinámicamente.
 - Existen pocas variantes estables y la herencia resulta más simple y comprensible.
 - La acumulación de decoradores produciría demasiados objetos pequeños y dificultaría comprender el comportamiento final.
- Decorator es útil cuando necesitamos combinar responsabilidades alrededor de un objeto sin crear una subclase para cada combinación.

Decorator

Actividad en clase: ¿Decorator?

Caso 1 — Documentos digitales

Un sistema genera documentos que inicialmente pueden visualizarse y descargarse. Dependiendo del contexto, algunos documentos deben incorporar marca de agua, firma digital, cifrado o registro de auditoría.

Estas características pueden utilizarse individualmente o combinarse.

¿Cómo diseñarían el sistema para incorporar diferentes combinaciones de funcionalidades sin crear una clase distinta para cada una?

Caso 2 — Personajes de videojuego

Un personaje posee un comportamiento básico, pero durante el juego puede equiparse con armas, armaduras, escudos y objetos especiales.

Cada elemento modifica o amplía determinadas capacidades y pueden utilizarse varias combinaciones.

¿Cómo permitirían agregar y combinar estas capacidades durante la ejecución sin definir una subclase para cada configuración posible?

Caso 3 — Procesamiento de imágenes

Una aplicación permite trabajar con una imagen original y añadir opcionalmente marca de agua, borde, compresión, ajuste de contraste o filtros.

El usuario puede aplicar una o varias operaciones en diferentes combinaciones.

¿Cómo diseñarían la solución para combinar estas responsabilidades manteniendo una interfaz común para procesar la imagen?

Caso 4 — Flujo de datos

Una aplicación transmite información mediante un flujo básico. Dependiendo de la necesidad, el flujo puede incorporar compresión, cifrado, registro de actividad o validación.

Las funcionalidades deben poder combinarse sin modificar la implementación básica del flujo.

¿Cómo diseñarían el sistema para añadir estas capacidades progresivamente sin crear una clase por cada combinación?

Unidad 02.02

Retroalimentación y cierre

Qué vimos hoy

- Bridge: separa dos dimensiones de variación para que puedan evolucionar independientemente.
- Decorator: permite añadir y combinar responsabilidades alrededor de un objeto sin crear una subclase para cada combinación.
- Ambos favorecen la composición, pero resuelven problemas estructurales diferentes.
- Bridge organiza dos jerarquías independientes conectadas mediante composición.
- Decorator mantiene una interfaz común y permite encadenar objetos que añaden responsabilidades.
- Un patrón debe aplicarse únicamente cuando el problema de diseño justifica la abstracción adicional.

Antes de finalizar:

- ¿Qué evidencia indica que un problema podría necesitar Bridge?
- ¿Qué evidencia indica que un problema podría necesitar Decorator?
- ¿Qué diferencia existe entre extender mediante herencia y extender mediante composición?
- ¿Qué riesgo aparece si utilizamos estos patrones sin necesidad?

Idea final: Bridge separa variaciones independientes; Decorator combina responsabilidades. La estructura puede parecer compleja, pero la intención del patrón debe ser siempre la que justifique su uso.

Anexos

Bibliografía

Gamma, E., Helm, R., Johnson, R. y Vlissides, J.; Design Patterns: Elements of Reusable Object-Oriented Software; 1.ª edición, 1994.

Sarcar, V.; Java Design Patterns: A Hands-On Experience with Real-World Examples; 3.ª edición, 2022.

Musch, O.; Design Patterns with Java: An Introduction; 1.ª edición, 2023.

Fowler, M.; UML Distilled: A Brief Guide to the Standard Object Modeling Language; 3.ª edición, 2004.

Shvets, A.; Sumérgete en los patrones de diseño; 1.ª edición, 2019.

IEEE Computer Society; Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a; versión 4.0a, 2025.

Asaad, J. y Avksentieva, E.; Assessing the Impact of GoF Design Patterns on Software Engineering Practices; 2025.

Anexos

Links de Referencia

- <https://www.campusmvp.es/recursos/post/los-conceptos-fundamentales-sobre-programacion-orientada-objetos-explicados-de-manera-simple.aspx>
- <https://codideep.com/blogpost/stupid-vs-solid>
- <https://refactoring.guru/design-patterns>
- <https://reactiveprogramming.io/books/design-patterns/es>
- <http://codejavu.blogspot.com/2013/07/ejemplo-patron-de-diseno-decorator.html>

Gracias por su atención

posgrados.ups.edu.ec